Create a client application that analyzes text

9 minutes

A **client application** is a program you write that connects to a service or model and uses its capabilities. Your code sends requests to the service and receives results back automatically — making it possible to process large volumes of text or integrate AI analysis into a workflow.

To connect to an AI service, your application uses an **API** (Application Programming Interface). An API is a set of rules that defines how two pieces of software communicate. A client library is a set of ready made code that developers can use in their application to easily talk to a service or API. You can review foundational material on applications and using endpoints in: Get started with AI in Azure.

Using general-purpose AI models for text analysis

Start with a Microsoft Foundry resource and create a Foundry project within your resource. In the *new* Foundry portal, you can browse the model catalog and deploy a general-purpose model.

You can build a client application that interacts with Microsoft Foundry Models using the **Azure OpenAI API**. The OpenAI API lets your code talk to a deployed model by sending requests to an *endpoint*, along with an *API key* to prove you're authorized.

The **Responses API** is the modern, unified API within Azure OpenAI for interacting with language models. It is designed to handle complete AI interactions, not just text generation.

You can use the *responses API* to send natural language prompts to a deployed language model. It's useful when you need flexible, conversational-style analysis that doesn't require a fixed structured output.

Using the OpenAI Python library

The *OpenAI Python library* is an official Python software development kit (SDK) that lets developers build Python applications that interact with OpenAI models and services through code instead of raw HTTP requests.

To use the OpenAI Python library, you need to work within a code editor. Application code is written in *code editors*, such as Visual Studio Code. A code editor's *terminal* is a built-in command-line window inside the editor where you can run commands without leaving your development environment.

1. Install the necessary packages

The OpenAI Python library can be installed in the Visual Studio Code *terminal* using:

Bash

```
pip install openai
```

2. Create a configuration file

Next, you can create a configuration file (type `.env`) to store your environment variables, such as your endpoint, key, and model deployment name.

Consider the following variables:

```
AZURE_OPENAI_ENDPOINT=https://<your-resource>.openai.azure.com/openai/v1/
MODEL_DEPLOYMENT_NAME=gpt-4.1-mini
API_KEY=<your-foundry-key>
```

Notice how the endpoint variable contains the name of your Foundry resource and `openai.azure.com/openai/v1`. Your API key is your Foundry project key.

The model deployment name is the name *you give* the model when you deploy it. For example, when you deploy the *gpt-4.1* model, you may name it *gpt-demo-model*. The deployment name is *gpt-demo-model*. However, if you do not customize the model name, the deployment name will match the model name, as is the case in the snippet above.

3. Create a file containing your application logic

Take a look at the following application code sample:

Python

```python
import os
from dotenv import load_dotenv
from openai import OpenAI

# Load environment variables from .env file
load_dotenv()
endpoint = os.getenv("AZURE_OPENAI_ENDPOINT")
api_key = os.getenv("API_KEY")
deployment_name = os.getenv("MODEL_DEPLOYMENT_NAME")

# Create the client object
client = OpenAI(
    base_url=endpoint,
    api_key=api_key
)
```

```
# Make a request using the client
message = client.responses.create(
    model=deployment_name,
    input="",
)

# Print the results
print(f"Sentiment: {message.output[0]}")
```

> **Note**
>
> **Loading environment variables**: In this sample, `dotenv` (`load_dotenv()`) reads your `.env` file and loads those values into your app's environment. The `os` package then retrieves each value by name with `os.getenv()`, such as `os.getenv("AZURE_OPENAI_ENDPOINT")`.
>
> Each key in `.env` must match the name in your code exactly. For example, if your file uses `API_KEY`, your code must also request `API_KEY`. Keep variable names consistent to avoid missing values at runtime.

We use our Foundry resource endpoint and key to create an authenticated **client object**. The `OpenAI` class is defined by the SDK and acts as a blueprint for connecting to the OpenAI API. An authenticated client object in Python is a service specific object that can securely make authorized API calls without your code manually managing tokens or secrets.

> **Note**
>
> In Python, a **class** is a blueprint that defines a type of thing — what data it holds and what actions it can perform. An **object** is a specific instance created from that blueprint. For example, a `Car` class might define that every car has a color and can `drive()` or `stop()`. When you create a specific car — say, a red one — that's an object.

Once you create a **client object** — configured with your endpoint and key — you can call **methods** on it to interact with the model. For example, you can use the `responses` *method* to send a prompt to a specific model deployment.

We can display the results of the analysis by running the application code in the terminal with the command `python <file_name>.py`.

The OpenAI API is straightforward to use, but results can vary between calls because the model generates text probabilistically. In practice, this means two calls with the same prompt can return slightly different wording or formatting. When your app needs consistent, structured values, such as a language code, confidence score, or redacted text, the Azure Language SDK is a better choice.

Using the Azure Language SDK

The **Azure Language SDK** is a client library for *Azure Language in Foundry Tools*. The SDK makes it easy for developers to add NLP features, such as language detection and redacting personally identifiable information (PII), to their applications.

Let's see how you can use the Azure Language Python SDK to build an application that analyzes text. To use the Azure Language Python SDK, you need to have a *Foundry resource*. Then you need to install a compatible version of Python and the Azure Language Python SDK.

The Python SDK can be installed in the Visual Studio Code *terminal* using:

Bash

```
pip install azure-ai-textanalytics
```

Consider the following configuration file sample:

Python

```
AZURE_LANGUAGE_ENDPOINT=https://<your-resource>.cognitiveservices.azure.com/
API_KEY=<your-foundry-key>
```

Consider the following application code sample:

Python

```python
# Import packages
import os
from dotenv import load_dotenv
from azure.core.credentials import AzureKeyCredential
from azure.ai.textanalytics import TextAnalyticsClient

# Load environment variables from .env file
load_dotenv()
endpoint = os.getenv("AZURE_LANGUAGE_ENDPOINT")
key = os.getenv("API_KEY")

# Create the client
client = TextAnalyticsClient(endpoint=endpoint, credential=AzureKeyCredential(key))

# Make a request using the client for language detection
text = "¡Hola! Me llamo Josefina y vivo en Madrid, España."
result = client.detect_language([text])[0]
```

We use the client's *methods* to call **Azure Language functions**, such as `detect_language` and `recognize_pii_entities`.

Language detection: The `detect_language()` method takes a list of text strings and returns the detected language, its ISO 639-1 code, and a confidence score between 0 and 1.

Python

```
text = "¡Hola! Me llamo Josefina y vivo en Madrid, España."
result = client.detect_language([text])[0]

# Print the results
print(f"Language      : {result.primary_language.name}")
print(f"ISO code      : {result.primary_language.iso6391_name}")
print(f"Confidence    : {result.primary_language.confidence_score:.2f}")
```

PII detection: The `recognize_pii_entities()` method identifies personal details in text and returns both the redacted version of the text and a list of the entities it found, including each entity's category and confidence score.

Python

```
text = "Maria Garcia called from 020 7946 0958 and asked to send documents to 42 Market Road, London, UK, SW1A 1AA."

result = client.recognize_pii_entities([text])[0]

# Print the results
print("Redacted text:", result.redacted_text)
print("\nEntities found:")
for entity in result.entities:
    print(f"  {entity.text} | category={entity.category} | confidence={entity.confidence_score}")
```

With the OpenAI API and the Azure Language SDK, you can write code for AI applications that process natural language and generate insight from your text.

Next, let's take a look at how to include Azure Language capabilities in AI agents.

Next unit: Use Azure Language with an agent

Previous Next